DROWSINESS DETECTION AND ALERTING SYSTEM

A PROJECT REPORT

Submitted by

JAYSHIVA.T (Team Leader) (Roll No:20URCS015)

MUTHUMARI.M (Roll No: 20URCS030)

KARTHIKA.R (Roll No: 20UTCS002)

TEAM ID : NM2023TMID01439

in partial fulfillment for the award of the

degree of

BACHELOR OF ENGINEERING

IN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AAA COLLEGE OF ENGINEERING AND TECHNOLOGY

ABSTRACT

ABSTRACT

This abstract presents a drowsiness detection and alerting system that utilizes the Eye Aspect Ratio (EAR) technique and OpenCV. The system continuously monitors the eye behavior of individuals in real-time using a webcam or similar camera. By calculating the EAR, which measures eye openness, the system accurately detects drowsiness. The EAR is computed by analyzing the Euclidean distances between specific eye points, providing a reliable measure of eye openness. In the system's main loop, video frames are continuously analyzed to determine the EAR. If the calculated EAR falls below a predetermined threshold (e.g., 0.25), indicating drowsiness, the system triggers an alert. An alert message is displayed on the screen, and an audio message is delivered using pyttsx3. This drowsiness detection and alerting system offers a cost-effective and non-intrusive solution for various applications, including driver monitoring systems and workplace safety.

TABLE OF CONTENTS

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	**INTRODUCTION**	**9**
	1.1 PROJECT OVERVIEW	**9**
	1.2 PURPOSE	**9**
2	**IDEATION & PROPOSED SOLUTION**	**12**
	2.1 PROBLEM STATEMENT DEFINITION	**12**
	2.2 EMPATHY MAP	**13**
	2.3 IDEATION & BRAINSTROMING	**15**
	2.4 PROPOSED SOLUTION	**18**

3	**REQUIREMENT ANALYSIS**	**20**
	3.1 FUNCTIONAL REQUIREMENT	**20**
	3.2 NON - FUNCTIONAL REQUIREMENT	**21**
4	**PROJECT DESIGN**	**24**
	4.1 DATA FLOW DIAGRAMS	**24**
	4.2 SOLUTION & TECHNICAL ARCHITECTURE	**25**
	4.3 USER STORIES	**27**
5	**CODING & SOLUTIONING**	**30**
	5.1 FEATURE 1: CREATE AND CONFIGURE IBM CLOUD SERVICES	**30**
	5.2 FEATURE 2: CREATE AND ACCESS NODE-RED	**35**

6	**RESULTS**	**39**
	6.1 PERFORMANCE METRICS	**39**
7	**ADVANTAGES & DISADVANTAGES**	**45**
8	**CONCLUSION**	**48**
9	**FUTURE SCOPE**	**50**
10	**APPENDIX**	**52**

INTRODUCTION

CHAPTER 1

INTRODUCTION

The introduction about the Drowsiness detection and alerting system are briefly discussed in this chapter.

1.1 PROJECT OVERVIEW

The project aims to develop a drowsiness detection and alerting system using the Eye Aspect Ratio (EAR) technique and OpenCV. The system focuses on detecting signs of drowsiness in individuals, particularly drivers, to enhance road safety and prevent accidents caused by driver fatigue.The system utilizes a webcam or a compatible camera to capture live video of the person's face. By leveraging OpenCV's face detection capabilities and the dlib library for facial landmark prediction, the system accurately identifies and tracks the position of the eyes within the video frames .In the system's main loop, video frames are continuously processed to calculate the EAR. When the EAR falls below a predetermined threshold, indicating drowsiness, the system triggers an alert. An alert message is displayed on the screen to notify the individual, while an audio message is delivered using the pyttsx3 library to provide an immediate warning. The system's integration of EAR and OpenCV enables real-time monitoring, timely alerts, and accurate detection of drowsiness, reducing the risk of potential accidents and promoting safer driving practices. The project holds significant potential for implementation in various transportation sectors and workplace environments where drowsiness poses a serious threat to human safety.

1.2 PURPOSE

Enhance Road Safety: The project aims to improve road safety by detecting and alerting individuals, particularly drivers, about their drowsiness levels. By addressing driver fatigue, it aims to prevent accidents caused by drowsy driving.

Real-time Monitoring: The system continuously monitors the eye behavior of individuals in real-time using a webcam or similar camera. This allows for prompt detection of drowsiness and immediate alerts to mitigate potential risks.

Timely Alerts: When the calculated EAR falls below a predefined threshold, indicating drowsiness, the system triggers alerts. These alerts are displayed on the screen and delivered through audio messages, ensuring that individuals receive timely
warnings to take necessary actions.

IoT Integration: The project incorporates IBM Watson IoT platform for remote monitoring and analysis of drowsiness status. By transmitting data to the platform, it enables comprehensive monitoring, data analysis, and potential integration with other IoT devices or systems.

Cost-effective Solution: The system utilizes commonly available hardware components, such as webcams and computers, making it a cost-effective solution for drowsiness detection. This opens up possibilities for widespread adoption and implementation in various domains.

IDEATION & PROPOSED SOLUTION

CHAPTER 2

IDEATION & PROPOSED SOLUTION

2.1 PROBLEM STATEMENT DEFINITION

Here, In my Project Drowsiness detection and Alerting System, A driver is a customer who needs to achieve his goal of safely taking all the passengers at correct time to their destination. Sometimes it's a difficult task for them to drive at night time because they may feel drowsy and exhausted without proper sleeping routines. It may cause accidents due to their drowsiness. To avoid this kind of accidents, We are trying to implement a Drowsiness Detection and Alerting System to monitor driver's eye movements by using Cameras and alerting him by using various means ,such as audible warnings, vibrations, or visual alerts.

Iam	a Driver
Iam trying to	Safely drop my passengers to their respective destination within expected time. I need to stay alert during night travels without feeling drowsy and fatigue inorder to avoid accidents and I need to follow strict traffic rules to avoid accidents. Iam responsible in ensuring that my vehicle is proper in condition for safe travel.
But	Sometimes I feel drowsy and sleepy as I drive many trips a day. This is the major problem faced by drivers who travel at night without rest. It may leadto road accidents or It may also cause health hazards for drivers too.
because	Its important for drivers to get enough sleep before driving at night. Needto stay hydrated at regular intervals.
Which makes me feel	Exhausted and drowsy

Table 2.1 Problem Statement Definition

Fig 2.1 Problem Statement Definition

2.2 EMPATHY MAP

An empathy map is a simple, easy –to-digest visual that captures knowledge about a user's behaviors and attitudes. It is a useful tool to helps teams better understand their users..

Creating an effective solution requires understanding the true problem and the person who is experiencing It.The exercise of creating the map helps participants consider things from the user's perspective along with his or her goals and challenge.

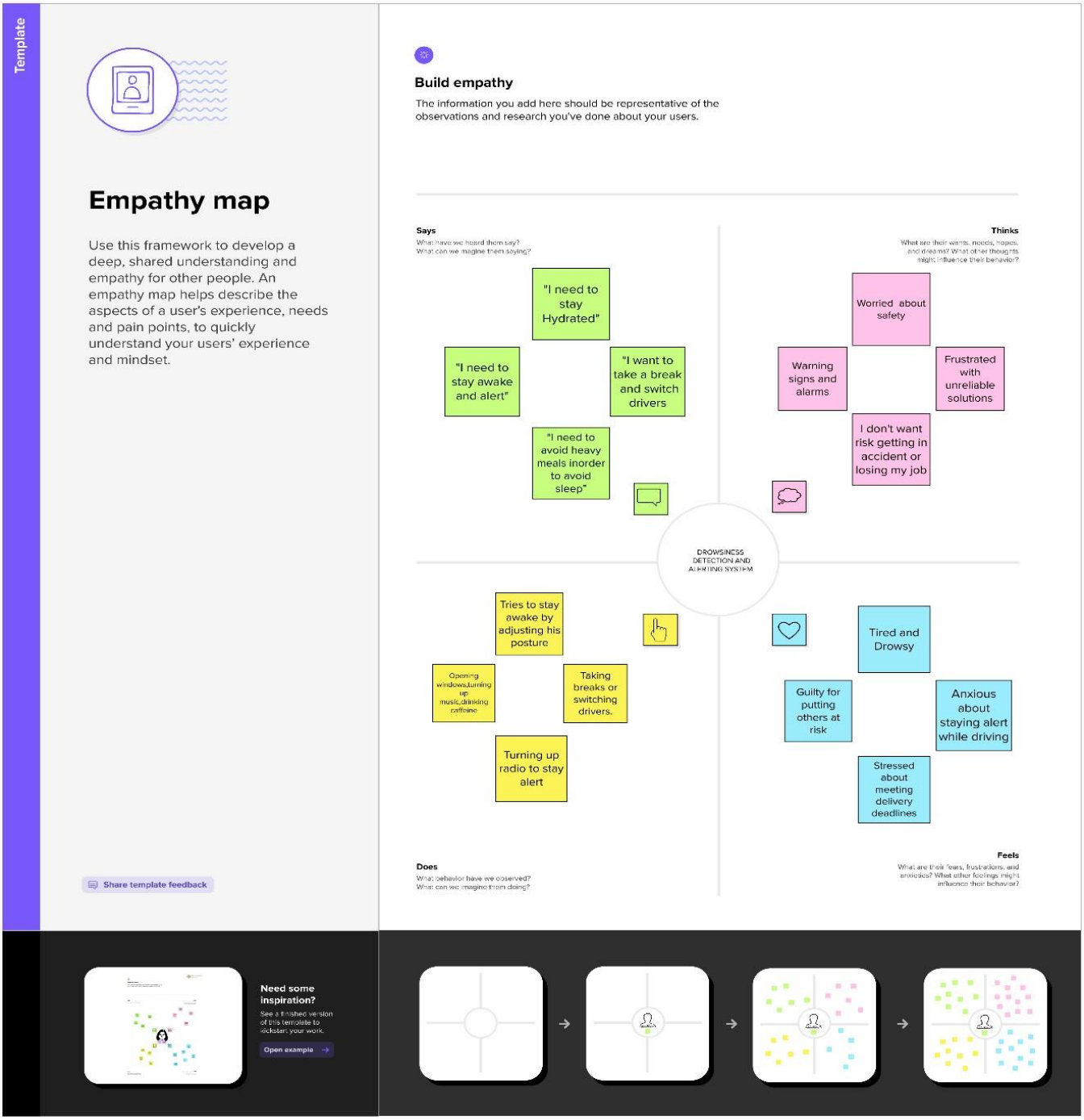


Fig 2.2 Empathy Map

2.3 IDEATION & BRAINSTORMING

Brainstorming provides a free and open environment that encourages everyone within a team to participate in the creative thinking process that leads to problem solving. Prioritizing volume over value, out-of-the-box ideas are welcome and built upon, and all participants are encouraged to collaborate, helping each other develop a rich amount of creative solutions.

Step-1: Team Gathering, Collaboration and Select the Problem Statement

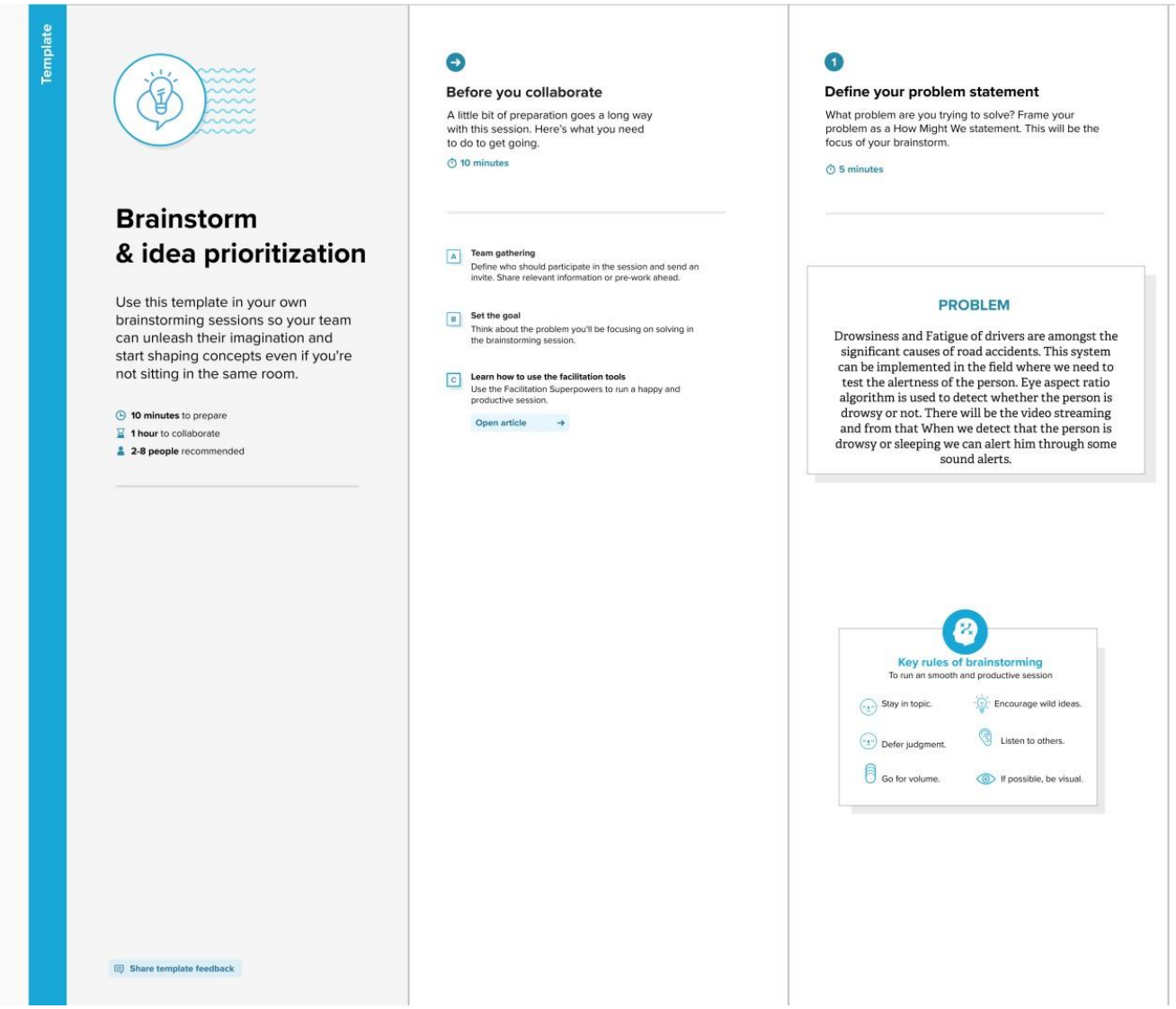


Fig.2.3 Brainstorming 1

Step-2: Brainstorm, Idea Listing and Grouping

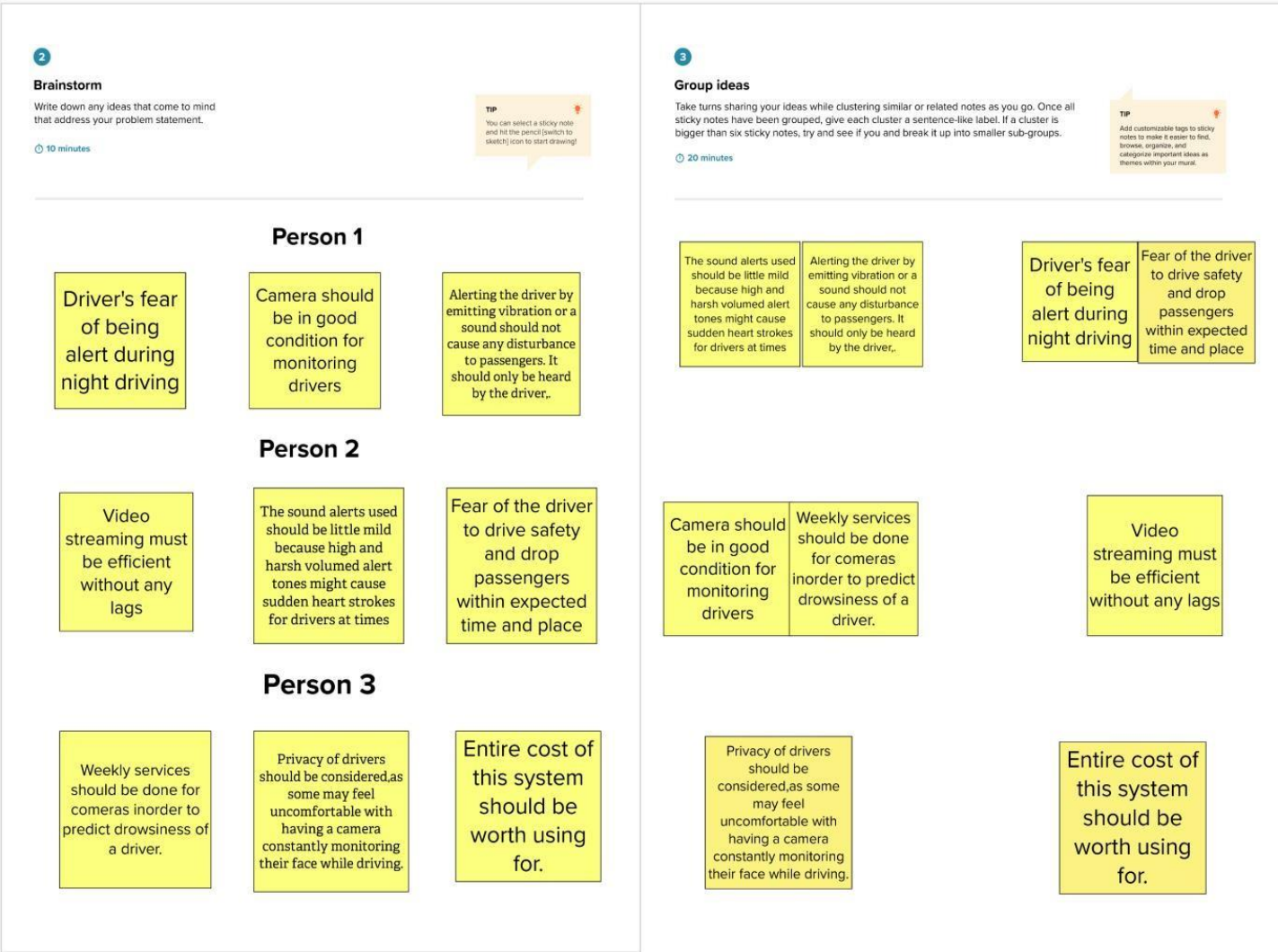


Fig 2.3 Brainstorming 2

Step-3: Idea Prioritization

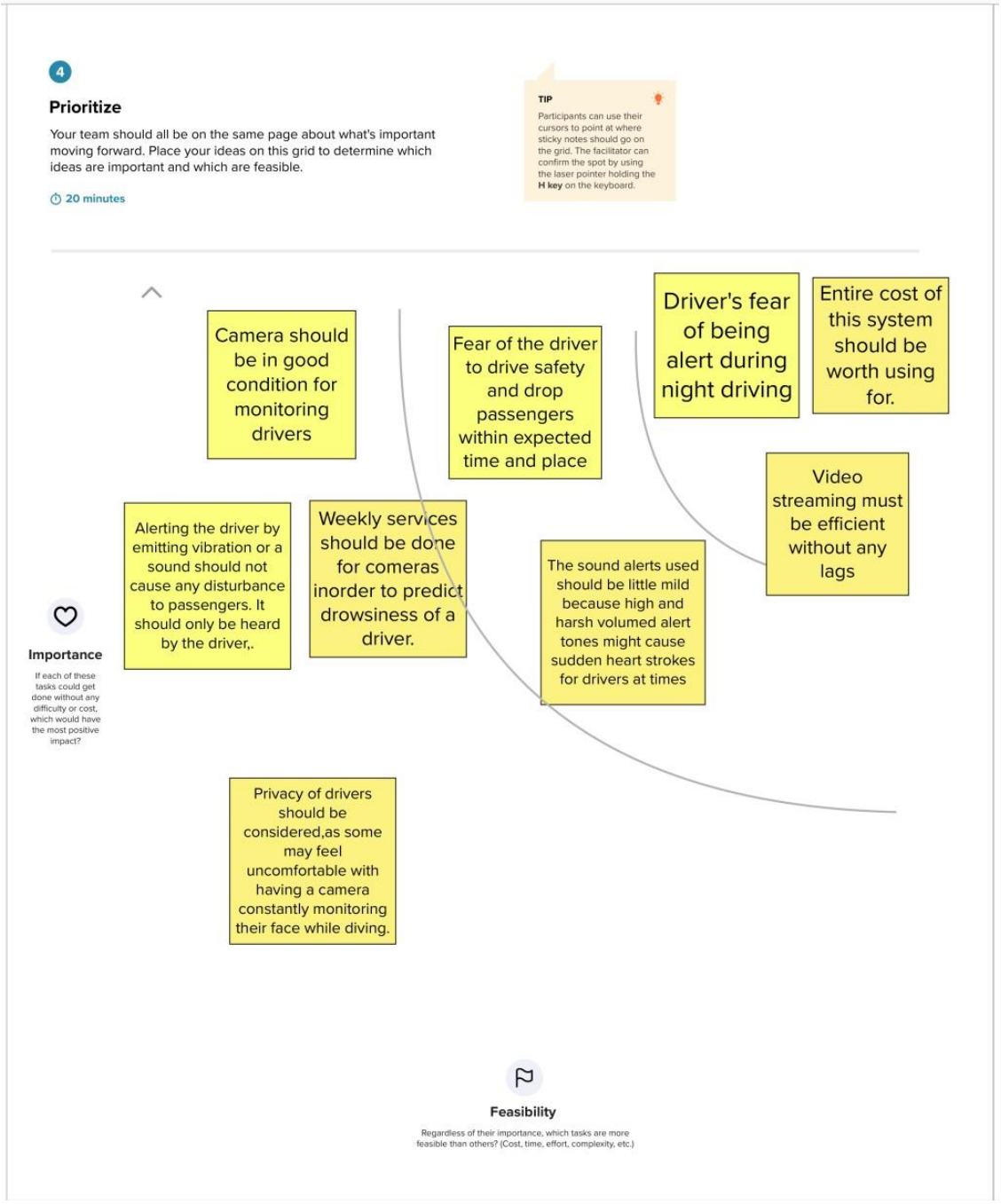


Fig 2.3 Brainstorming 3

2.4 PROPOSED SOLUTION

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	The problem to be solved is to prevent accidents caused by drowsiness or fatigue whiledriving or operating heavy machinery. This is a critical safety issue as drowsy driving can lead to accidents that can cause injuries, fatalities, and property damage.
2.	Idea / Solution description	Our proposed solution is to develop a drowsiness detection and alerting system whichtests the alertness of the driver. Eye aspect ratio algorithm is used to detect whether the person is drowsy or not. There will be the videostreaming and from that When we detect that the person is drowsy or sleeping we can alert him through some sound alerts.
3.	Novelty / Uniqueness	Our proposed solution is unique in using machine learning algorithms and the usage of EAR (Eye Aspect Ratio) as a metric for drowsiness detection is that it can be easily calculated using a standard camera, making it a cost-effective solution compared to other drowsiness detection systems that may require specialized sensors or equipment.
4.	Social Impact / Customer Satisfaction	The system will improve road safety by reducing the number of accidents caused by drowsy driving. It will also increase customer satisfaction by providing an additional support for the safety of drivers.
5.	Business Model (Revenue Model)	The system can be sold as a standalone productor integrated into other products, such as in- vehicle alert systems or wearable devices. Revenue is generated through the sale of the product.
6.	Scalability of the Solution	Our solution is highly scalable as it can be easily integrated into existing vehicles and Machine learning algorithms can help improve the scalability of the system by automating the analysis and decision-making process.

Table 2.4 Proposed solution

REQUIREMENT ANALYSIS

CHAPTER 3

REQUIREMENT ANALYSIS

In this chapter, the requirement analysis of the proposed system has been discussed along with the brief explanation about its advantages.

3.1 FUNTIONAL REQUIREMENT

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Alert Generation	• Generate audible warnings, such as beeps or voice prompts, to alert the driver. • Implement haptic feedback, such as vibrating seats or steering wheels, to provide physical alerts.
FR-2	Customizable Settings	• Develop a user interface for drivers to adjust sensitivity levels or response parameters. • Allow customization of alert types, such as visual, auditory, or haptic alerts, based on driver preferences. • Enable the modification of alert intensities to accommodate individual driving styles or tolerances.
FR-3	Real-time Monitoring	• Capture and analyze driver's eye movements, including blink rate and duration of eye closure. • Monitor head movements and changes in facial expressions to identify signs of drowsiness.

FR-4	Integration with Vehicle Systems	• Ensure compatibility with vehicle-specific interfaces and communication protocols.
FR-5	System Maintenance	• To identify sensor failures or system malfunctions. • Implement regular system checks
FR-6	Compliance with Safety Standards	• Perform validation and testing procedures to meet certification requirements. • Keep up-to-date with evolving industry standards and best practices for driver drowsiness detection systems.

3.2 NON-FUNCTIONAL REQUIREMENT

Following are the non-functional requirements of the proposed solution.

FR No.	**Non-Functional Requirement**	**Description**
NFR-1	**Usability**	• The system should have a user-friendly interface for both drivers and fleet managers. • The alerts and warnings should be easily understandable.
NFR-2	**Security**	• The system should employ measures to protect the privacy and confidentiality of driver data. • The system should meet relevant safety standards and regulations.
NFR-3	**Reliability**	• The system should consistently and accurately detect signs of drowsiness or fatigue. • The system should be robust and resistant to environmental factors that may affect its performance.

NFR-4	**Performance**	• The system should have low latency in detecting drowsiness and generating alerts. • The system should handle high volumes of data in real-time without significant delays.
NFR-5	**Availability**	• The system should strive for high uptime, minimizing downtime due to maintenance, upgrades, or unexpected failures.
NFR-6	**Scalability**	• The system should be able to accommodate a large number of vehicles and drivers in a fleet. • The system should handle increasing data volume without compromising performance.

This chapter dealt with the functional and non-functional requirement analysis of proposed system.

PROJECT DESIGN

CHAPTER 4

PROJECT DESIGN

4.1 DATA FLOW DIAGRAMS

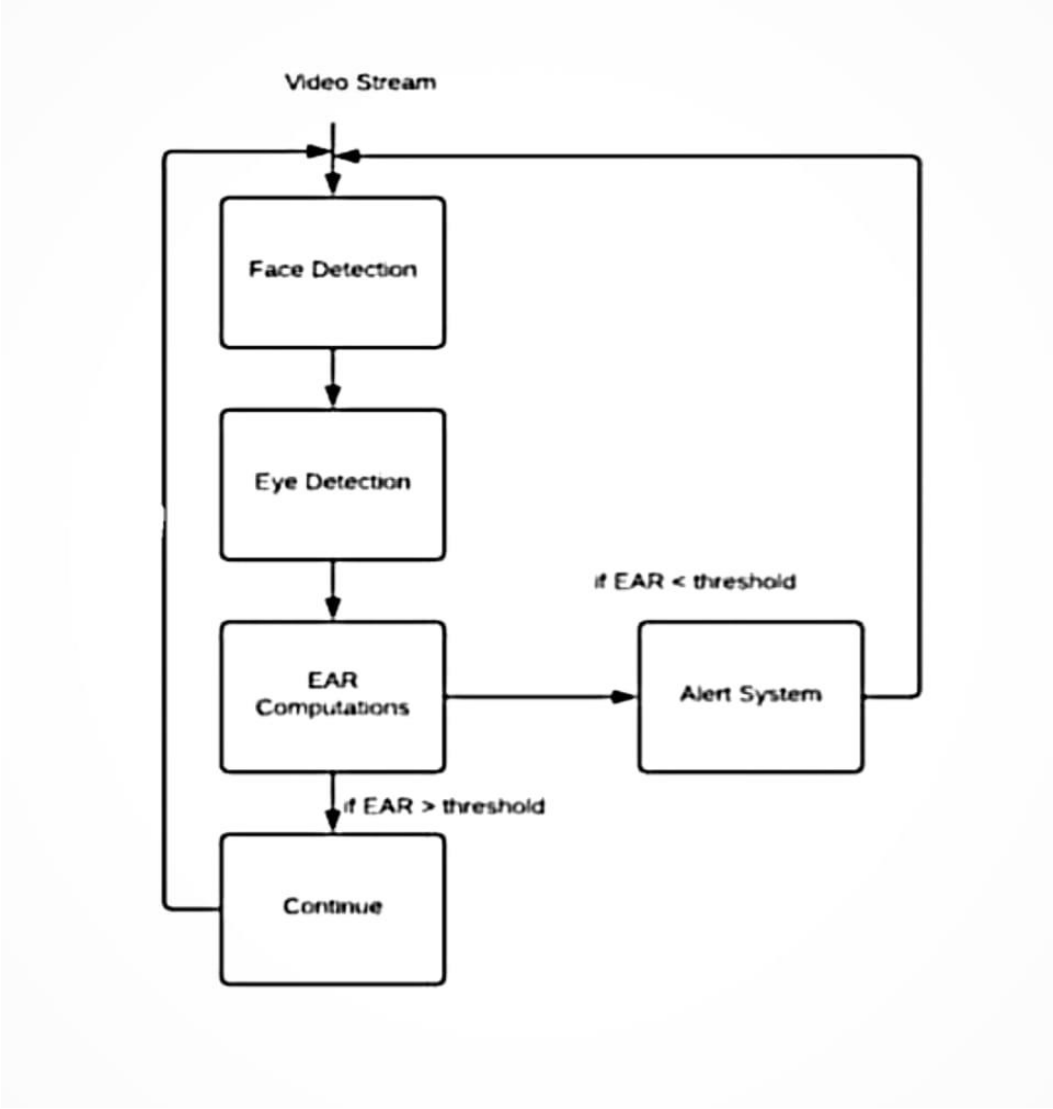


Fig 4.1 Dataflow Diagram

4.1 SOLUTION & TECHNICAL ARCHITECTURE

4.1.1 SOLUTION ARCHITECTURE

Image Acquisition: The system acquires images of the driver's face through acamera, which is placed in a suitable position to capture the face

Feature Extraction: The images are processed to extract facial features such aseye landmarks, which are used to calculate the eye aspect ratio (EAR)

EAR Calculation: The EAR is calculated in real-time using the eye landmarks, andis used as a metric for drowsiness detection

Drowsiness Detection: The calculated EAR is compared to a threshold value to determine if the driver is drowsy. If the EAR falls below the threshold value, the system generates an alert to notify the driver

Alert Generation: The system generates an alert to notify the driver of their drowsiness. This alert can take the form of a sound, vibration, or other visual cue,depending on the implementation of the system.

Data Storage: The data from the system, such as images and EAR values, can bestored for further analysis or monitoring purposes.

System Management: The system can be managed through a user interface or control module, which allows the user to configure settings, view data, and performother management functions.

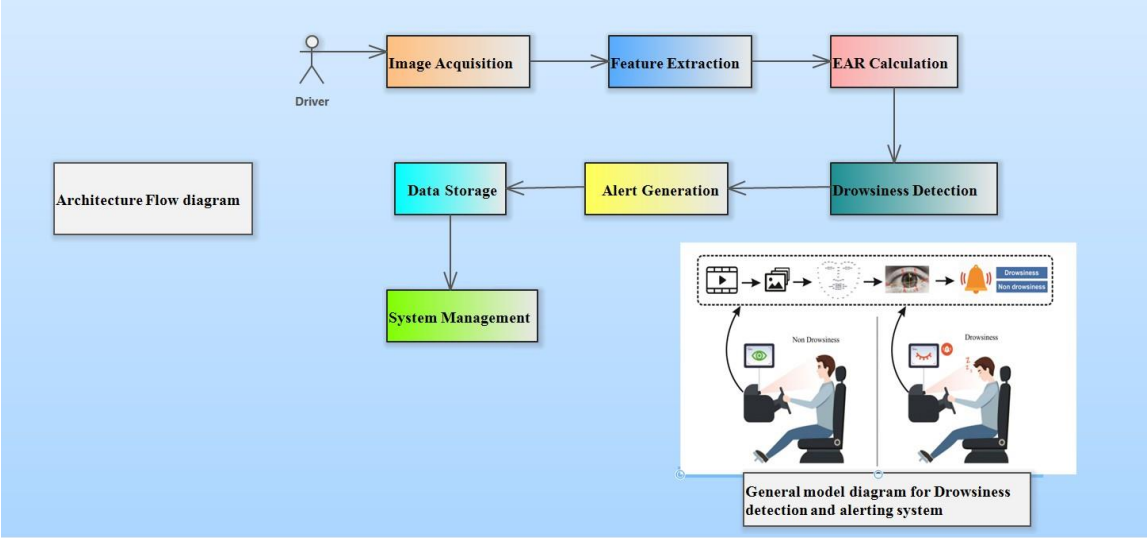


Fig. 4.2.2. Solution architecture

4.2.3 TECHNICAL ARCHITECTURE

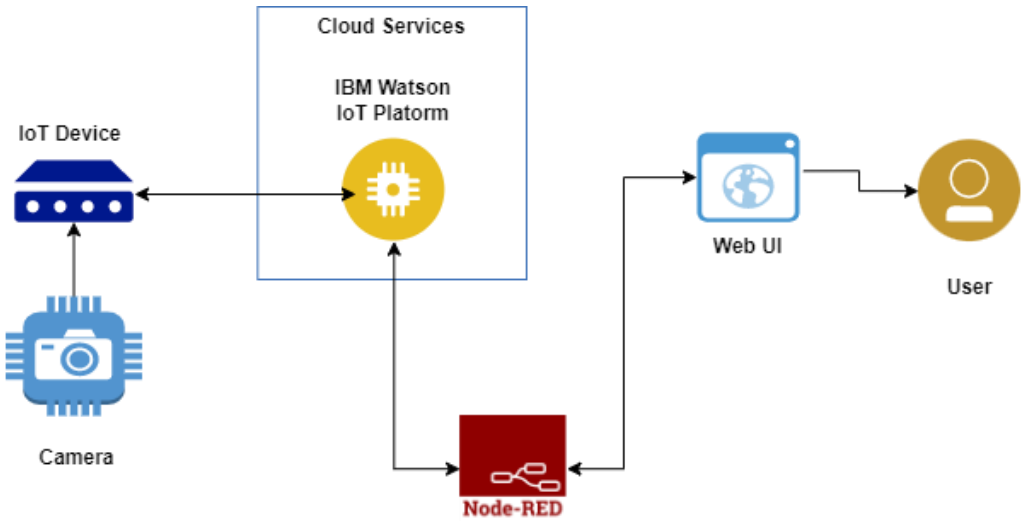


Fig 4.2.3 Technical architecture

4.3 USER STORIES

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
1.Driver	Alert Generation	USN-1	As a driver, I want to be alerted when signs of drowsiness or fatigue are detected so that I can take necessary measures to stay awake and ensure safe driving.	I need to respond to the alerts of the system	High	Sprint-1
	Customizable Settings	USN-2	As a driver, I should have the ability to customize the sensitivity and response of the drowsiness detection system according to their preferences or specific requirements.	I need to access the system's settings menu and modify parameters such as alert thresholds or warning types based on individual requirements.	Medium	Sprint-1
	Regular Maintenance	USN-3	As a driver ,I should follow recommended maintenance procedures for the drowsiness detection system to ensure its proper functioning.	I need to adhere to the manufacturer's guidelines for periodic system checks, software updates, or hardware maintenance, if applicable.	High	Sprint-2
2.Fleet Managers	System Implementation	USN-1	As a Fleet manager,I should identify and implement a drowsiness detection and alerting system for the fleet.	I need to select a reliable system that meets the fleet's requirements in terms of accuracy, compatibility with existing vehicles, and ease of integration.	High	Sprint-1
	Driver Training and Awareness	USN-2	As a Fleet manager,I should train drivers on the proper use and importance of the drowsiness detection and alerting system.	To conduct training sessions or provide instructional materials to ensure drivers understand how	High	Sprint-2

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
				the system works, how to respond to alerts.		
	System Maintenance and Upgrades	USN-3	As a Fleet manager, I ensure the proper functioning of the drowsiness detection system through maintenance and periodic updates.	Implement a maintenance schedule to perform system checks, software updates, and address any hardware issues, if applicable, to ensure optimal performance.	Medium	Sprint-2
3.Researchers and Developers	Research and Analysis	USN-1	As a Researcher , I should conduct research on drowsiness detection techniques, algorithms, and technologies and analyze existing research and studies related to drowsiness detection and driver behavior.	Conduct a comprehensive literature review on drowsiness detection techniques and studies. Identify relevant algorithms, sensors, and technologies to explore and incorporate into the system.	High	Sprint-1
	System Architecture and Design	USN-2	As a Researcher , I should Design the overall architecture of the drowsiness detection system, considering modularity, scalability, and performance.	Documenting and sharing research findings with the team and presenting a summary of relevant algorithms.	Medium	Sprint-2

CODING AND SOLUTIONING

CHAPTER 5

CODING AND SOLUTIONING

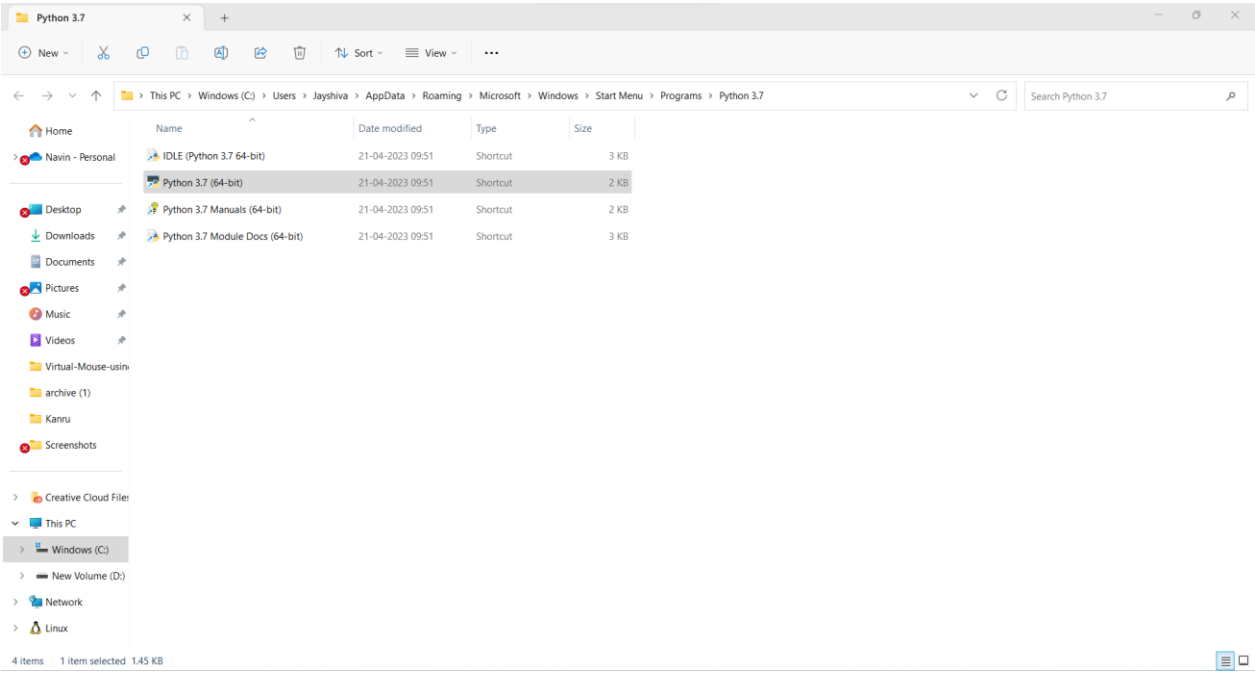
5.1 FEATURE 1: CREATE AND CONFIGURE IBM CLOUD SERVICES

STEP 1: The initial step is to create an IBM account.

STEP 2: After creating cloud account , launch IBM IOT Watson platform by accessing cloud account.

STEP 3: **Running the source code in Python shell**

STEP 3.1: Install the Python Software (Version 3.7.0)



STEP 3.2: Develop the python code to publish details to IBM IoT Platform.

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
import cv2
import dlib
import requests
import pyttsx3
from scipy.spatial import distance
import time
import sys
import ibmiotf.application
import ibmiotf.device
import random

#Provide your IBM Watson Device Credentials
organization = "2br5s5"
deviceType = "test"
deviceId = "123321"
authMethod = "token"
authToken = "12345678"

def ibmstart(x):

  def myCommandCallback(cmd):
    print("Command received: %s" % cmd.data['command'])
    print(cmd)

  try:
   deviceOptions = {"org": organization, "type": deviceType, "id": deviceId, "auth-method": authMethod, "auth-token": authToken}
   deviceCli = ibmiotf.device.Client(deviceOptions)
   #..............................................

  except Exception as e:
   print("Caught exception connecting device: %s" % str(e))
   sys.exit()
```

STEP 3.3: Integrate our own IBM Watson Credentials to connect to our Device in IBM IoT Platform.

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

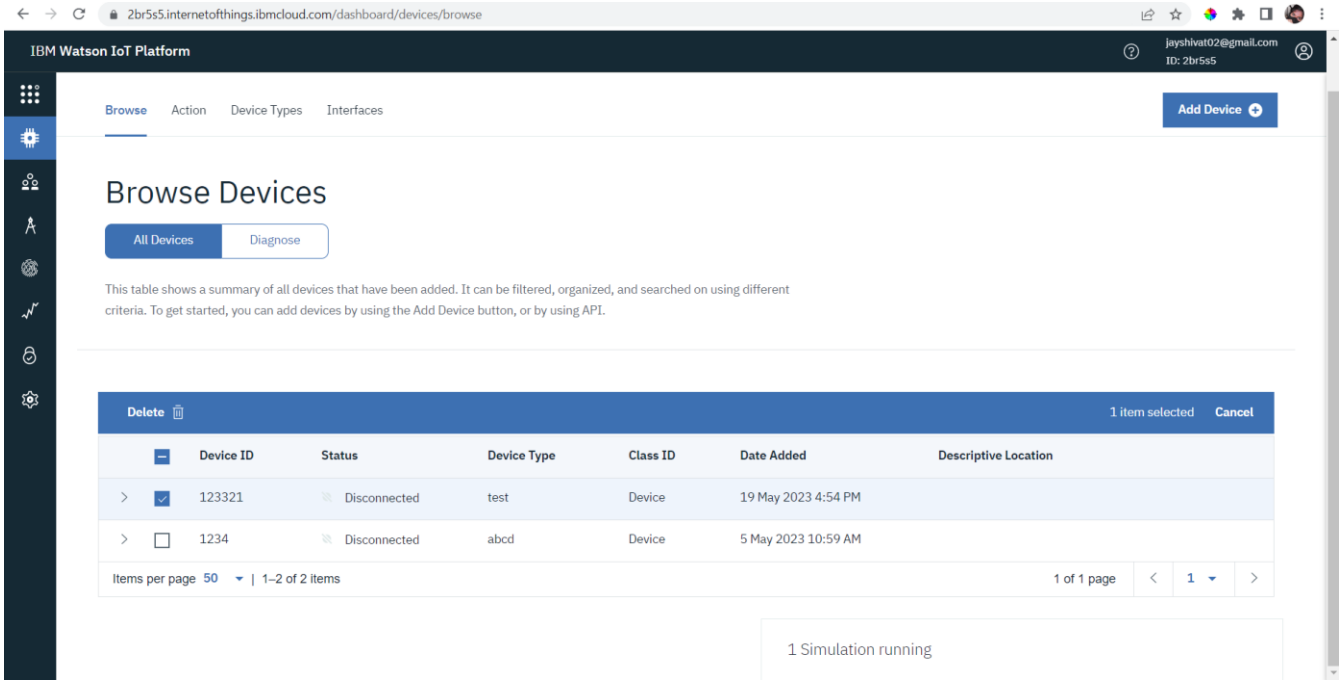
File Edit Format Run Options Window Help

```
import cv2
import dlib
import requests
import pyttsx3
from scipy.spatial import distance
import time
import sys
import ibmiotf.application
import ibmiotf.device
import random

#Provide your IBM Watson Device Credentials
organization = "2br5s5"
deviceType = "test"
deviceId = "123321"
authMethod = "token"
authToken = "12345678"

def ibmstart(x):

  def myCommandCallback(cmd):
    print("Command received: %s" % cmd.data['command'])
    print(cmd)

  try:
   deviceOptions = {"org": organization, "type": deviceType, "id": deviceId, "auth-method": authMethod, "auth-token": authToken}
   deviceCli = ibmiotf.device.Client(deviceOptions)
   #..............................................

  except Exception as e:
   print("Caught exception connecting device: %s" % str(e))
   sys.exit()
```

After running the source code successfully in Python shell , the results should be published to IBM IoT Watson Platform.

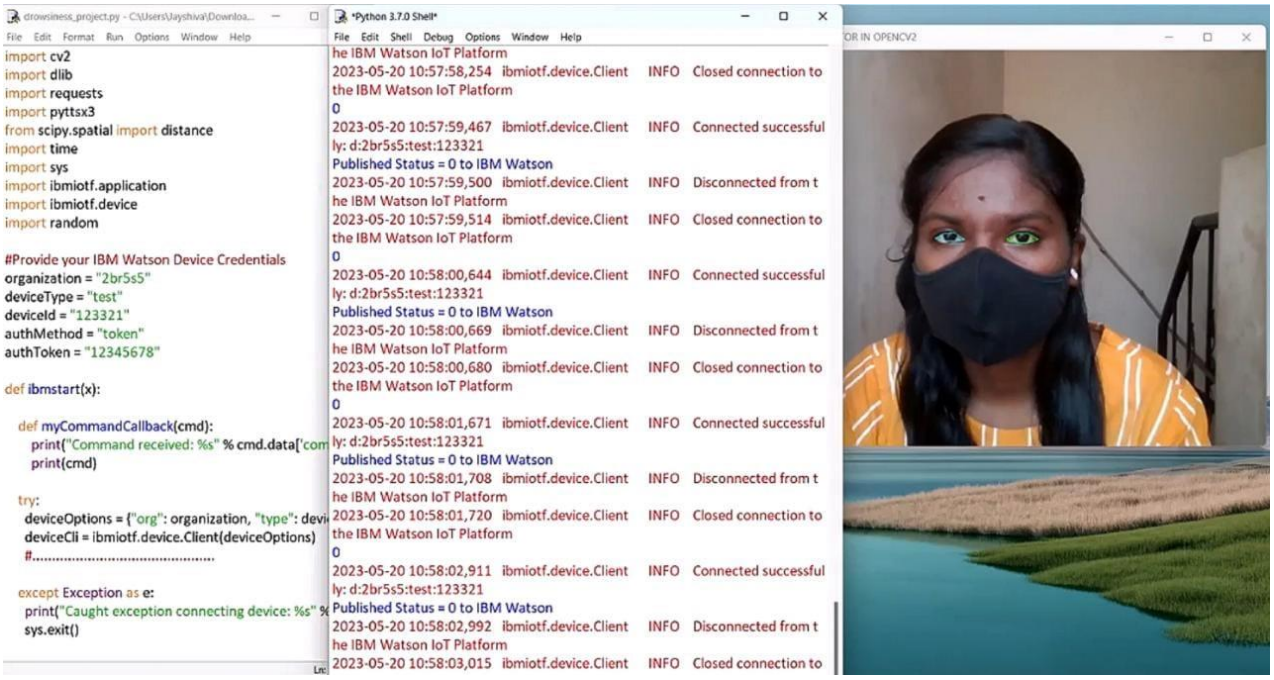
STEP 4: Integration with IBM Watson IoT Platform

STEP 4.1.1 : Creation of a Device

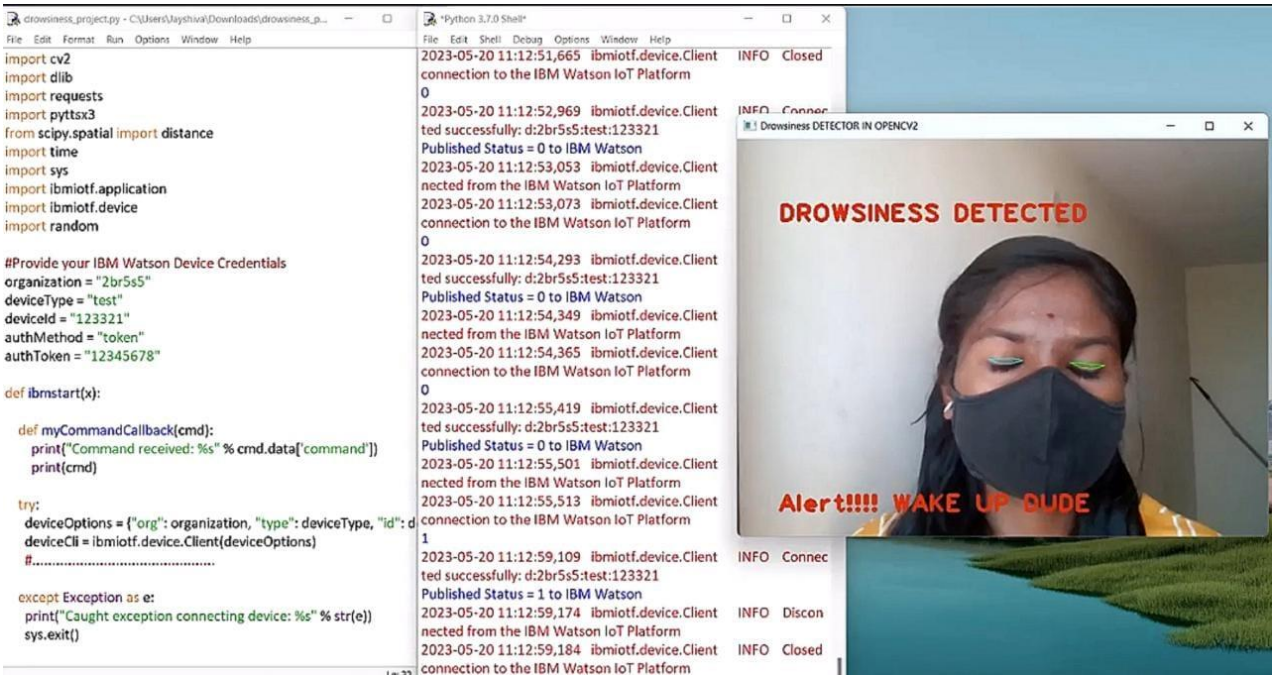


STEP 5: After getting the required output,

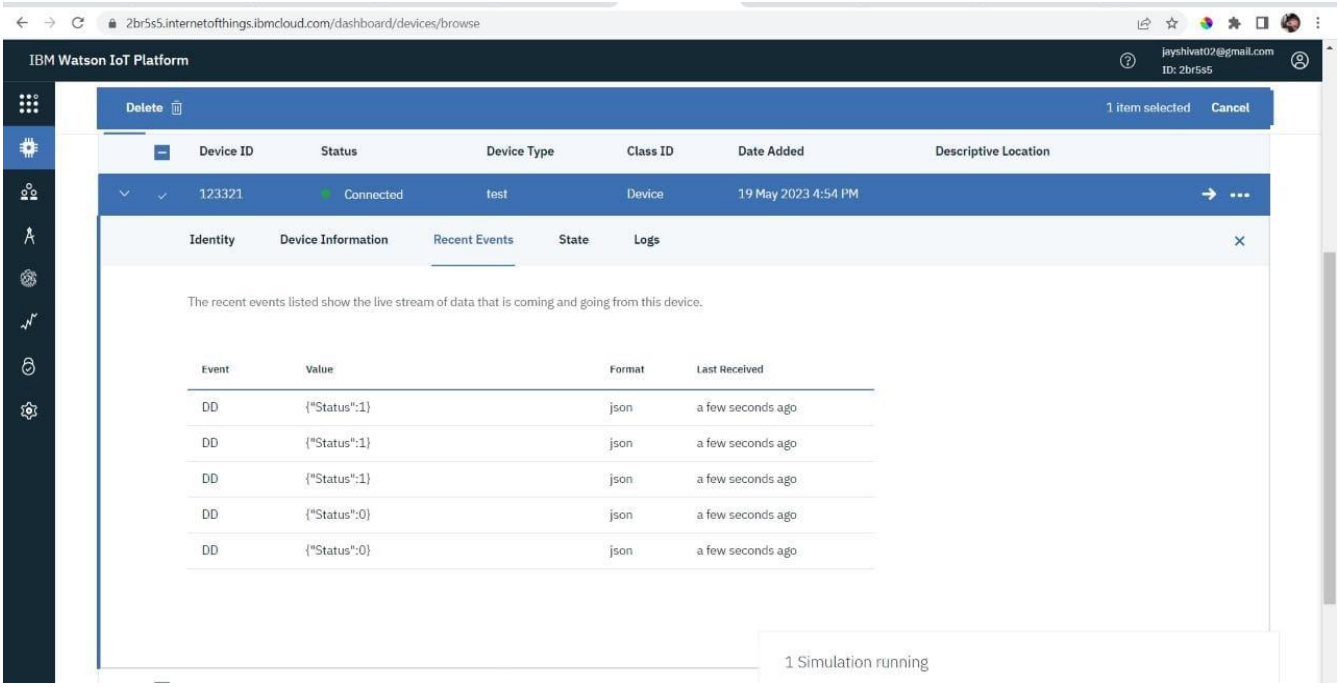
STEP 5.1 : When the person is not Drowsy, The Status will be published as 0



STEP 5.2 : When the person feels Drowsy, The Status will be published as 1 and an Alert message is Displayed on the screen and an Audio is also played to alert the person.

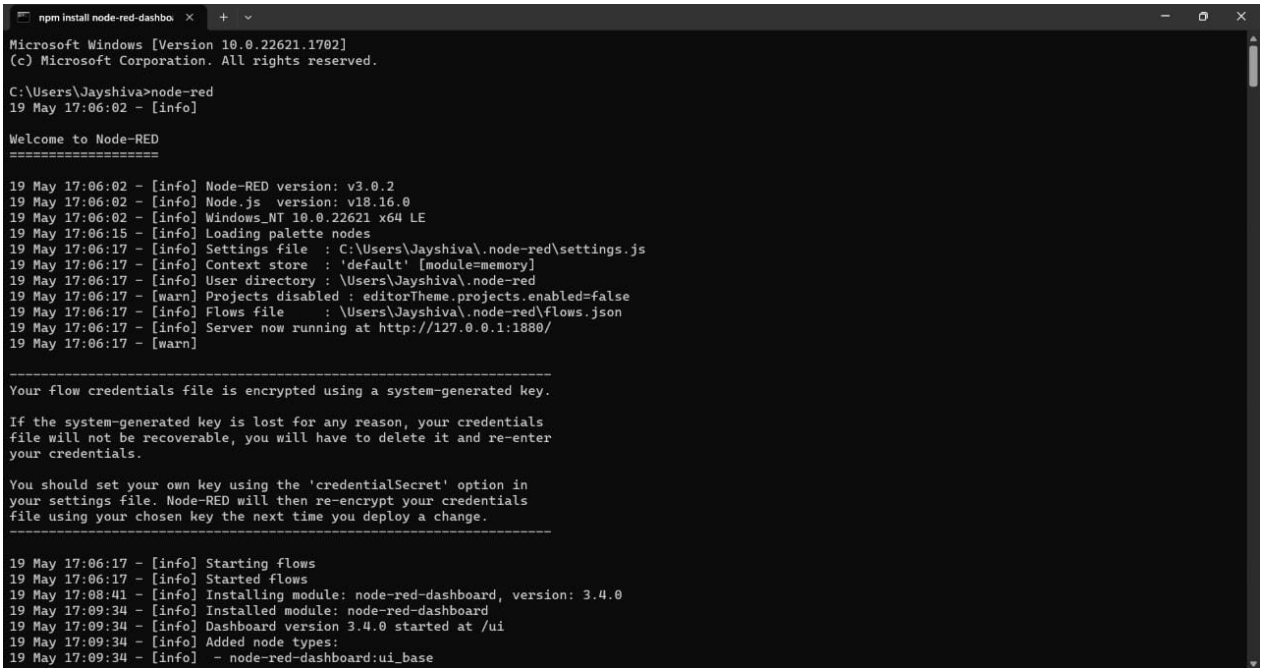


After the connection is Made successfully, The program output is published to IBM Watson IoT Platform in the Recent Events section .



5.2 FEATURE 2: CREATE AND ACCESS NODE-RED

STEP 1: Node-Red Installation

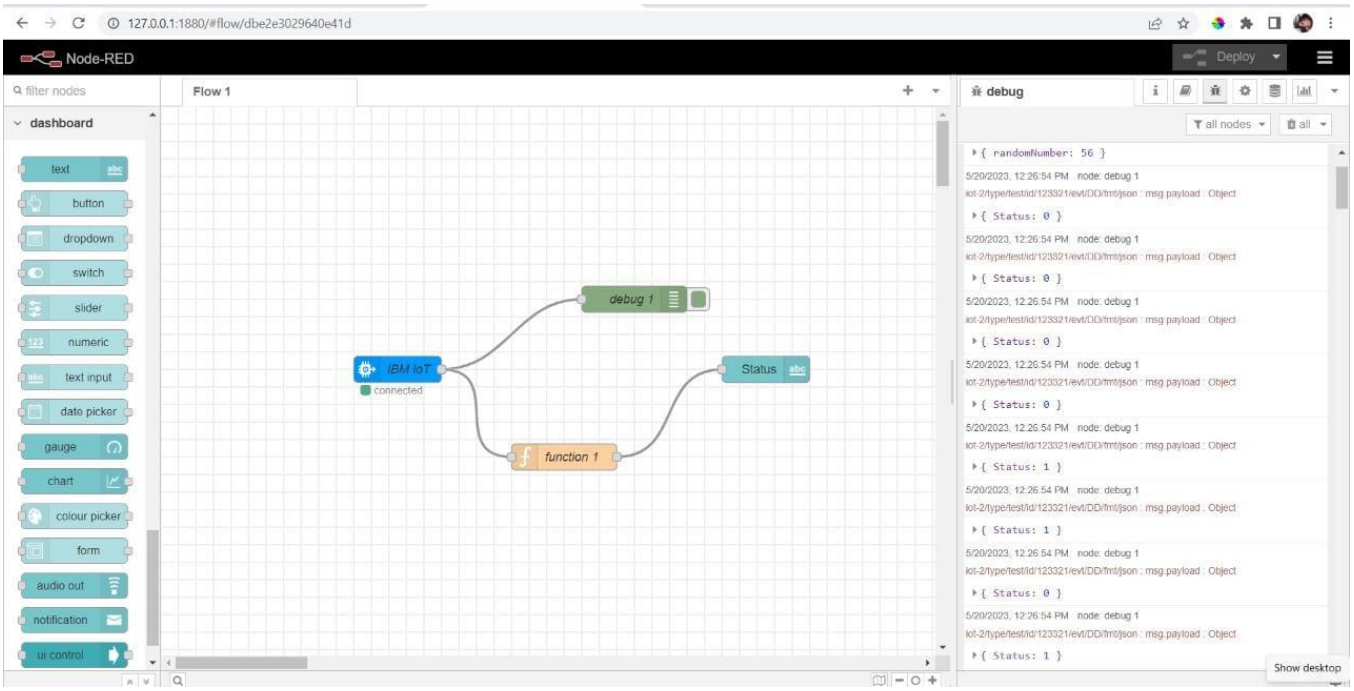


```
npm install node-red-dashbo
Microsoft Windows [Version 10.0.22621.1702]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Jayshiva>node-red
19 May 17:06:02 - [info]

Welcome to Node-RED
===================

19 May 17:06:02 - [info] Node-RED version: v3.0.2
19 May 17:06:02 - [info] Node.js  version: v18.16.0
19 May 17:06:02 - [info] Windows_NT 10.0.22621 x64 LE
19 May 17:06:15 - [info] Loading palette nodes
19 May 17:06:17 - [info] Settings file  : C:\Users\Jayshiva\.node-red\settings.js
19 May 17:06:17 - [info] Context store  : 'default' [module=memory]
19 May 17:06:17 - [info] User directory : \Users\Jayshiva\.node-red
19 May 17:06:17 - [warn] Projects disabled : editorTheme.projects.enabled=false
19 May 17:06:17 - [info] Flows file     : \Users\Jayshiva\.node-red\flows.json
19 May 17:06:17 - [info] Server now running at http://127.0.0.1:1880/
19 May 17:06:17 - [warn]

---------------------------------------------------------------------
Your flow credentials file is encrypted using a system-generated key.

If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
---------------------------------------------------------------------

19 May 17:06:17 - [info] Starting flows
19 May 17:06:17 - [info] Started flows
19 May 17:08:41 - [info] Installing module: node-red-dashboard, version: 3.4.0
19 May 17:09:34 - [info] Installed module: node-red-dashboard
19 May 17:09:34 - [info] Dashboard version 3.4.0 started at /ui
19 May 17:09:34 - [info] Added node types:
19 May 17:09:34 - [info]  - node-red-dashboard:ui_base
```

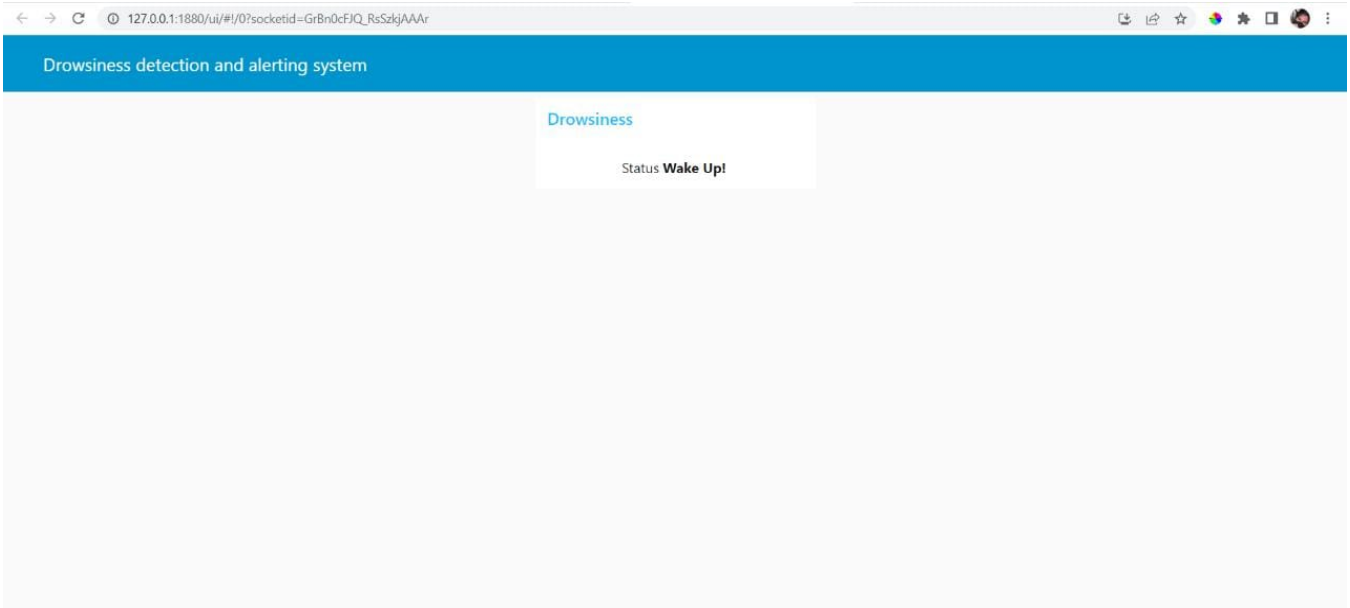
STEP 2: Connect IBM Watson with Node-red through API key

STEP 3: Design the project flow using Node-Red

STEP 4: Check for the proper connections and the output is obtained in the node red application.



STEP 5: When the Dashboard is Opened in Node-red, It displays the status message of the person.



RESULTS

CHAPTER 6

RESULTS

6.1 PERFORMANCE METRICS

CODE:

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
import cv2
import dlib
import requests
import pyttsx3
from scipy.spatial import distance
import time
import sys
import ibmiotf.application
import ibmiotf.device
import random

#Provide your IBM Watson Device Credentials
organization = "2br5s5"
deviceType = "test"
deviceId = "123321"
authMethod = "token"
authToken = "12345678"

def ibmstart(x):

  def myCommandCallback(cmd):
    print("Command received: %s" % cmd.data['command'])
    print(cmd)

  try:
   deviceOptions = {"org": organization, "type": deviceType, "id": deviceId, "auth-method": authMethod, "auth-token": authToken}
   deviceCli = ibmiotf.device.Client(deviceOptions)
   #..............................................

  except Exception as e:
   print("Caught exception connecting device: %s" % str(e))
   sys.exit()
```

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
   sys.exit()

  deviceCli.connect()
  data = { 'Status' : x}
  #print data
  def myOnPublishCallback():
    print ("Published Status = %s" % x, "to IBM Watson")

  success = deviceCli.publishEvent("DD", "json", data, qos=0, on_publish=myOnPublishCallback)
  if not success:
    print("Not connected to IoTF")


  deviceCli.commandCallback = myCommandCallback
  deviceCli.disconnect()
# INITIALIZING THE pyttsx3 SO THAT
# ALERT AUDIO MESSAGE CAN BE DELIVERED
engine = pyttsx3.init()

# SETTING UP OF CAMERA TO 1 YOU CAN
# EVEN CHOOSE 0 IN PLACE OF 1
cap = cv2.VideoCapture(0)

# FACE DETECTION OR MAPPING THE FACE TO
# GET THE Eye AND EYES DETECTED
face_detector = dlib.get_frontal_face_detector()

# PUT THE LOCATION OF .DAT FILE (FILE FOR
# PREDECTING THE LANDMARKS ON FACE )
dlib_facelandmark = dlib.shape_predictor("C:\Kanru\shape_predictor_68_face_landmarks.dat")
```

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
# PUT THE LOCATION OF .DAT FILE (FILE FOR
# PREDECTING THE LANDMARKS ON FACE )
dlib_facelandmark = dlib.shape_predictor("C:\Kanru\shape_predictor_68_face_landmarks.dat")

# FUNCTION CALCULATING THE ASPECT RATIO FOR
# THE Eye BY USING EUCLIDEAN DISTANCE FUNCTION
def Detect_Eye(eye):
 poi_A = distance.euclidean(eye[1], eye[5])
 poi_B = distance.euclidean(eye[2], eye[4])
 poi_C = distance.euclidean(eye[0], eye[3])
 aspect_ratio_Eye = (poi_A+poi_B)/(2*poi_C)
 return aspect_ratio_Eye


# MAIN LOOP IT WILL RUN ALL THE UNLESS AND
# UNTIL THE PROGRAM IS BEING KILLED BY THE USER
while True:

 null, frame = cap.read()
 flag=0
 gray_scale = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

 faces = face_detector(gray_scale)

 for face in faces:
  face_landmarks = dlib_facelandmark(gray_scale, face)
  leftEye = []
  rightEye = []

  # THESE ARE THE POINTS ALLOCATION FOR THE
  # LEFT EYES IN .DAT FILE THAT ARE FROM 42 TO 47
  for n in range(42, 48):
   x = face_landmarks.part(n).x
```

Ln: 91 Col: 0

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
  # THESE ARE THE POINTS ALLOCATION FOR THE
  # LEFT EYES IN .DAT FILE THAT ARE FROM 42 TO 47
  for n in range(42, 48):
   x = face_landmarks.part(n).x
   y = face_landmarks.part(n).y
   rightEye.append((x, y))
   next_point = n+1
   if n == 47:
    next_point = 42
   x2 = face_landmarks.part(next_point).x
   y2 = face_landmarks.part(next_point).y
   cv2.line(frame, (x, y), (x2, y2), (0, 255, 0), 1)

  # THESE ARE THE POINTS ALLOCATION FOR THE
  # RIGHT EYES IN .DAT FILE THAT ARE FROM 36 TO 41
  for n in range(36, 42):
   x = face_landmarks.part(n).x
   y = face_landmarks.part(n).y
   leftEye.append((x, y))
   next_point = n+1
   if n == 41:
    next_point = 36
   x2 = face_landmarks.part(next_point).x
   y2 = face_landmarks.part(next_point).y
   cv2.line(frame, (x, y), (x2, y2), (255, 255, 0), 1)

  # CALCULATING THE ASPECT RATIO FOR LEFT
  # AND RIGHT EYE
  right_Eye = Detect_Eye(rightEye)
  left_Eye = Detect_Eye(leftEye)
  Eye_Rat = (left_Eye+right_Eye)/2

  # NOW ROUND OF THE VALUE OF AVERAGE MEAN
```

Ln: 120 Col: 10

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
  # NOW ROUND OF THE VALUE OF AVERAGE MEAN
  # OF RIGHT AND LEFT EYES
  Eye_Rat = round(Eye_Rat, 2)

  # THIS VALUE OF 0.25 (YOU CAN EVEN CHANGE IT)
  # WILL DECIDE WHETHER THE PERSONS'S EYES ARE CLOSE OR NOT
  if Eye_Rat < 0.25:
   cv2.putText(frame, "DROWSINESS DETECTED", (50, 100),
       cv2.FONT_HERSHEY_PLAIN, 2, (21, 56, 210), 3)
   cv2.putText(frame, "Alert!!!! WAKE UP DUDE", (50, 450),
       cv2.FONT_HERSHEY_PLAIN, 2, (21, 56, 212), 3)

   # CALLING THE AUDIO FUNCTION OF TEXT TO
   # AUDIO FOR ALERTING THE PERSON
   engine.say("Alert!!!! WAKE UP DUDE")
   flag=1
   engine.runAndWait()

 cv2.imshow("Drowsiness DETECTOR IN OPENCV2", frame)
 print(flag)
 ibmstart(flag)
 '''
 while True:

        data = { 'Status' : x}
        #print data
        def myOnPublishCallback():
          print ("Published Status = %s" % x, "to IBM Watson")

        success = deviceCli.publishEvent("DD", "json", data, qos=0, on_publish=myOnPublishCallback)
        if not success:
          print("Not connected to IoTF")
        time.sleep(1)
```

Ln: 152 Col: 18

drowsiness_project.py - C:\Users\Jayshiva\Downloads\drowsiness_project.py (3.7.0)

File Edit Format Run Options Window Help

```
   engine.say("Alert!!!! WAKE UP DUDE")
   flag=1
   engine.runAndWait()

 cv2.imshow("Drowsiness DETECTOR IN OPENCV2", frame)
 print(flag)
 ibmstart(flag)
 '''
 while True:

        data = { 'Status' : x}
        #print data
        def myOnPublishCallback():
          print ("Published Status = %s" % x, "to IBM Watson")

        success = deviceCli.publishEvent("DD", "json", data, qos=0, on_publish=myOnPublishCallback)
        if not success:
          print("Not connected to IoTF")
        time.sleep(1)

        deviceCli.commandCallback = myCommandCallback
 '''
 #r1 = requests.get('https://api.thingspeak.com/update?api_key=SEWZDEK7APG3P0P8&field1='+str(flag))
 #print(r1.status_code)
 key = cv2.waitKey(9)
 if key == 20:
  break

# Disconnect the device and application from the cloud
#deviceCli.disconnect()
cap.release()
cv2.destroyAllWindows()
```

Ln: 167 Col: 0

OUTPUT:

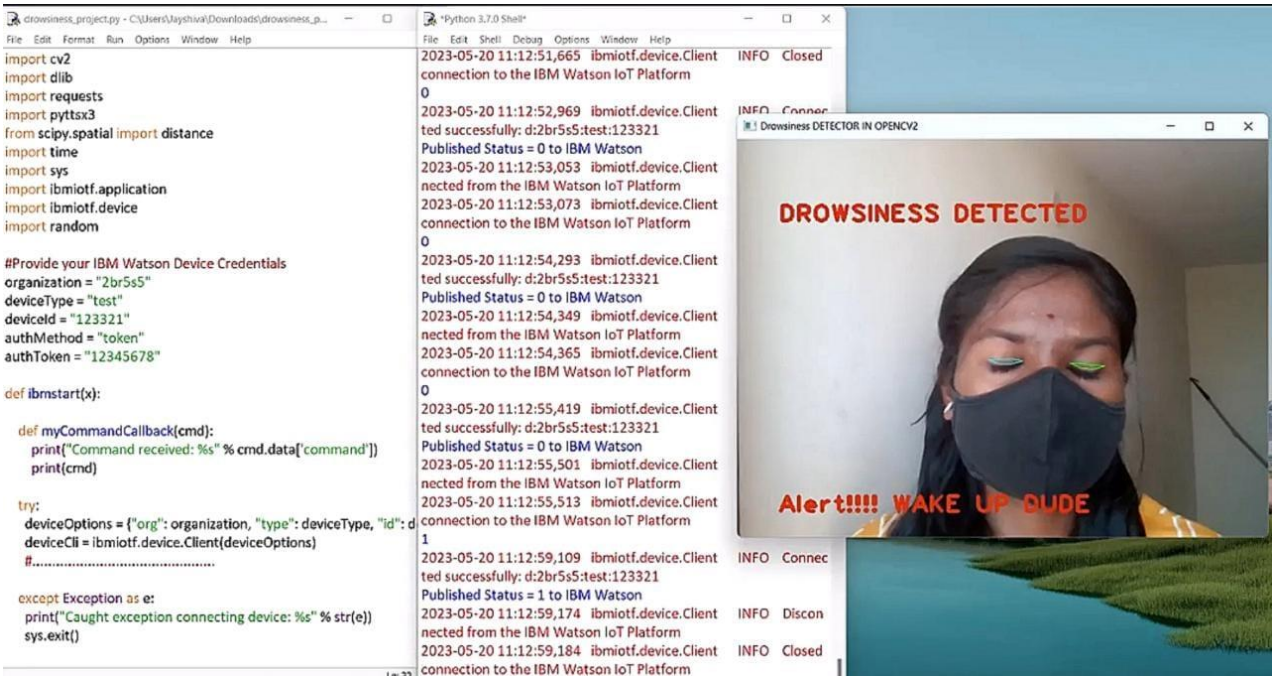
```
*Python 3.7.0 Shell*
File  Edit  Shell  Debug  Options  Window  Help
Published Status = 0 to IBM Watson
2023-05-20 11:48:11,861  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:11,865  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
0
2023-05-20 11:48:13,102  ibmiotf.device.Client   INFO  Connected successfully: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 11:48:13,138  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:13,143  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
0
2023-05-20 11:48:14,388  ibmiotf.device.Client   INFO  Connected successfully: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 11:48:14,468  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:14,481  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
0
2023-05-20 11:48:15,609  ibmiotf.device.Client   INFO  Connected successfully: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 11:48:15,639  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:15,650  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
0
2023-05-20 11:48:16,643  ibmiotf.device.Client   INFO  Connected successfully: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 11:48:16,699  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:16,709  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
0
2023-05-20 11:48:17,773  ibmiotf.device.Client   INFO  Connected successfully: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 11:48:17,785  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:17,790  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
0
2023-05-20 11:48:18,996  ibmiotf.device.Client   INFO  Connected successfully: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 11:48:19,028  ibmiotf.device.Client   INFO  Disconnected from the IBM Watson IoT Platform
2023-05-20 11:48:19,035  ibmiotf.device.Client   INFO  Closed connection to the IBM Watson IoT Platform
```

When the person is not drowsy,

```
drowsiness_project.py - C:\Users\Jayshiva\Downloa...
File  Edit  Format  Run  Options  Window  Help
import cv2
import dlib
import requests
import pyttsx3
from scipy.spatial import distance
import time
import sys
import ibmiotf.application
import ibmiotf.device
import random

#Provide your IBM Watson Device Credentials
organization = "2br5s5"
deviceType = "test"
deviceId = "123321"
authMethod = "token"
authToken = "12345678"

def ibmstart(x):

    def myCommandCallback(cmd):
        print("Command received: %s" % cmd.data['com
        print(cmd)

    try:
        deviceOptions = {"org": organization, "type": devi
        deviceCli = ibmiotf.device.Client(deviceOptions)
        #..............................................

    except Exception as e:
        print("Caught exception connecting device: %s" %
        sys.exit()
```

```
*Python 3.7.0 Shell*
File  Edit  Shell  Debug  Options  Window  Help
he IBM Watson IoT Platform
2023-05-20 10:57:58,254  ibmiotf.device.Client   INFO  Closed connection to
the IBM Watson IoT Platform
0
2023-05-20 10:57:59,467  ibmiotf.device.Client   INFO  Connected successful
ly: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 10:57:59,500  ibmiotf.device.Client   INFO  Disconnected from t
he IBM Watson IoT Platform
2023-05-20 10:57:59,514  ibmiotf.device.Client   INFO  Closed connection to
the IBM Watson IoT Platform
0
2023-05-20 10:58:00,644  ibmiotf.device.Client   INFO  Connected successful
ly: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 10:58:00,669  ibmiotf.device.Client   INFO  Disconnected from t
he IBM Watson IoT Platform
2023-05-20 10:58:00,680  ibmiotf.device.Client   INFO  Closed connection to
the IBM Watson IoT Platform
0
2023-05-20 10:58:01,671  ibmiotf.device.Client   INFO  Connected successful
ly: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 10:58:01,708  ibmiotf.device.Client   INFO  Disconnected from t
he IBM Watson IoT Platform
2023-05-20 10:58:01,720  ibmiotf.device.Client   INFO  Closed connection to
the IBM Watson IoT Platform
0
2023-05-20 10:58:02,911  ibmiotf.device.Client   INFO  Connected successful
ly: d:2br5s5:test:123321
Published Status = 0 to IBM Watson
2023-05-20 10:58:02,992  ibmiotf.device.Client   INFO  Disconnected from t
he IBM Watson IoT Platform
2023-05-20 10:58:03,015  ibmiotf.device.Client   INFO  Closed connection to
```

When the person feels drowsy,



ADVANTAGES AND DISADVANTAGES

CHAPTER 7

ADVANTAGES AND DISADVANTAGES

ADVANTAGES:

Drowsiness Detection: The project effectively detects drowsiness in individuals using computer vision techniques. It analyzes facial landmarks and eye aspect ratio to identify signs of drowsiness accurately.

Integration with IBM Watson Platform: The integration with the IBM Watson platform allows for seamless data transfer and storage of drowsiness events. It provides a reliable and secure infrastructure for storing and accessing the system's output data.

Alerting Mechanism: The project includes an alerting mechanism that triggers an audio message when drowsiness is detected. This feature can effectively notify the individual to wake up or take necessary actions to prevent accidents.

IBM Watson Cloud Connectivity: The project connects to the IBM Watson platform, enabling data visualization, monitoring, and analysis. It provides a centralized and scalable cloud infrastructure for managing the system's data and monitoring its performance.

DISADVANTAGES:

Hardware and Environmental Dependencies: The project relies on specific hardware components such as a camera and a computer system capable of running the necessary software libraries. It may not be easily deployable in environments where these hardware components are not readily available.

Single-Person Detection: The project is designed to detect drowsiness in individuals within the camera's field of view. It may not be suitable for scenarios where multiple individuals need to be monitored simultaneously.

Dependency on Facial Landmark Detection: The accuracy of the drowsiness detection heavily relies on the accurate detection of facial landmarks. Any limitations or inaccuracies in the facial landmark detection algorithm could affect the system's performance.

Sensitivity to Lighting and Environmental Conditions: The performance of the system can be influenced by lighting conditions, shadows, and environmental factors. Changes in lighting or adverse environmental conditions may impact the accuracy of drowsiness detection.

CONCLUSION

CHAPTER 8

CONCLUSION

In conclusion, the project successfully developed a drowsiness detection and alerting system using Python, OpenCV, IBM Watson, and Node-RED. The system demonstrated the capability to detect signs of drowsiness in individuals by analyzing facial landmarks and monitoring the eye aspect ratio in real-time.

Through the integration with IBM Watson, the project leveraged the cloud platform's capabilities for data storage, visualization, and monitoring. This integration provided a scalable and reliable infrastructure for managing and analyzing the drowsiness detection events.

FUTURE SCOPE

CHAPTER 9

FUTURE SCOPE

Multi-modal detection: Extend the system to detect drowsiness using multiple modalities, such as monitoring heart rate, body movements, or EEG signals. This can provide a more comprehensive and accurate assessment of drowsiness levels.

Personalized alerts: Tailor the alert mechanisms based on individual preferences and requirements. This could involve adjusting the intensity or type of alert based on the person's responsiveness or incorporating personalized audio or visual cues.

Driver assistance systems: Integrate the drowsiness detection system with existing driver assistance systems, such as adaptive cruise control or lane departure warning systems. This can enable the vehicle to autonomously react to detected drowsiness, like slowing down, activating alarms, or providing gentle vibrations.

Integration with wearables: Explore integration with wearable devices, such as smartwatches or fitness trackers, to monitor physiological signals and provide real-time drowsiness alerts. This can enable individuals to receive personalized alerts even outside the context of driving or operating machinery.

Compliance and regulations: As drowsiness detection systems become more advanced and widespread, there may be opportunities for regulatory guidelines and standards. Contributing to the development of industry standards and compliance frameworks can ensure the safe and effective use of such systems.

APPENDIX

CHAPTER 10

APPENDIX

Source Code:

```
import cv2
import dlib
import requests
import pyttsx3
from scipy.spatial import distance
import time
import sys
import ibmiotf.application
import ibmiotf.device
import random

#Provide your IBM Watson Device Credentials
organization = "2br5s5"
deviceType = "test"
deviceId = "123321"
authMethod = "token"
authToken = "12345678"

def ibmstart(x):

    def myCommandCallback(cmd):
        print("Command received: %s" % cmd.data['command'])
        print(cmd)

    try:
      deviceOptions = {"org": organization, "type": deviceType, "id": deviceId, "auth-method": authMethod,
"auth-token": authToken}
      deviceCli = ibmiotf.device.Client(deviceOptions)
      #..............................................

    except Exception as e:
      print("Caught exception connecting device: %s" % str(e))
      sys.exit()
```

```
    deviceCli.connect()
    data = { 'Status' : x}
    #print data
    def myOnPublishCallback():
        print ("Published Status = %s" % x, "to IBM Watson")
    success = deviceCli.publishEvent("DD", "json", data, qos=0, on_publish=myOnPublishCallback)
    if not success:
        print("Not connected to IoTF")

    deviceCli.commandCallback = myCommandCallback
    deviceCli.disconnect()

# INITIALIZING THE pyttsx3 SO THAT
# ALERT AUDIO MESSAGE CAN BE DELIVERED
engine = pyttsx3.init()

# SETTING UP OF CAMERA TO 1 YOU CAN
# EVEN CHOOSE 0 IN PLACE OF 1
cap = cv2.VideoCapture(0)

# FACE DETECTION OR MAPPING THE FACE TO
# GET THE Eye AND EYES DETECTED
face_detector = dlib.get_frontal_face_detector()

# PUT THE LOCATION OF .DAT FILE (FILE FOR
# PREDECTING THE LANDMARKS ON FACE )

dlib_facelandmark = dlib.shape_predictor("C:\Kanru\shape_predictor_68_face_landmarks.dat")

# FUNCTION CALCULATING THE ASPECT RATIO FOR
# THE Eye BY USING EUCLIDEAN DISTANCE FUNCTION
def Detect_Eye(eye):
  poi_A = distance.euclidean(eye[1], eye[5])
  poi_B = distance.euclidean(eye[2], eye[4])
  poi_C = distance.euclidean(eye[0], eye[3])
  aspect_ratio_Eye = (poi_A+poi_B)/(2*poi_C)
  return aspect_ratio_Eye
```

```
# MAIN LOOP IT WILL RUN ALL THE UNLESS AND
# UNTIL THE PROGRAM IS BEING KILLED BY THE USER
while True:

  null, frame = cap.read()
  flag=0
  gray_scale = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

  faces = face_detector(gray_scale)

  for face in faces:
    face_landmarks = dlib_facelandmark(gray_scale, face)
    leftEye = []
    rightEye = []

    # THESE ARE THE POINTS ALLOCATION FOR THE
    # LEFT EYES IN .DAT FILE THAT ARE FROM 42 TO 47
    for n in range(42, 48):
      x = face_landmarks.part(n).x
      y = face_landmarks.part(n).y
      rightEye.append((x, y))
      next_point = n+1
      if n == 47:
        next_point = 42
      x2 = face_landmarks.part(next_point).x
      y2 = face_landmarks.part(next_point).y
      cv2.line(frame, (x, y), (x2, y2), (0, 255, 0), 1)

    # THESE ARE THE POINTS ALLOCATION FOR THE
    # RIGHT EYES IN .DAT FILE THAT ARE FROM 36 TO 41

    for n in range(36, 42):
      x = face_landmarks.part(n).x
      y = face_landmarks.part(n).y
      leftEye.append((x, y))
      next_point = n+1
      if n == 41:
        next_point = 36
      x2 = face_landmarks.part(next_point).x
      y2 = face_landmarks.part(next_point).y
```

```
    cv2.line(frame, (x, y), (x2, y2), (255, 255, 0), 1)

  # CALCULATING THE ASPECT RATIO FOR LEFT
  # AND RIGHT EYE
  right_Eye = Detect_Eye(rightEye)
  left_Eye = Detect_Eye(leftEye)
  Eye_Rat = (left_Eye+right_Eye)/2

  # NOW ROUND OF THE VALUE OF AVERAGE MEAN
  # OF RIGHT AND LEFT EYES
  Eye_Rat = round(Eye_Rat, 2)

  # THIS VALUE OF 0.25 (YOU CAN EVEN CHANGE IT)
  # WILL DECIDE WHETHER THE PERSONS'S EYES ARE CLOSE OR NOT
  if Eye_Rat < 0.25:
    cv2.putText(frame, "DROWSINESS DETECTED", (50, 100),
        cv2.FONT_HERSHEY_PLAIN, 2, (21, 56, 210), 3)
    cv2.putText(frame, "Alert!!!! WAKE UP DUDE", (50, 450),
        cv2.FONT_HERSHEY_PLAIN, 2, (21, 56, 212), 3)

    # CALLING THE AUDIO FUNCTION OF TEXT TO
    # AUDIO FOR ALERTING THE PERSON
    engine.say("Alert!!!! WAKE UP DUDE")
    flag=1
    engine.runAndWait()

cv2.imshow("Drowsiness DETECTOR IN OPENCV2", frame)
print(flag)
ibmstart(flag)
'''
while True:

        data = { 'Status' : x}
        #print data
        def myOnPublishCallback():
            print ("Published Status = %s" % x, "to IBM Watson")

        success = deviceCli.publishEvent("DD", "json", data, qos=0, on_publish=myOnPublishCallback)
        if not success:
            print("Not connected to IoTF")
```

```
            time.sleep(1)

            deviceCli.commandCallback = myCommandCallback
  '''
  #r1 = requests.get('https://api.thingspeak.com/update?api_key=SEWZDEK7APG3P0P8&field1='+str(flag))
  #print(r1.status_code)
  key = cv2.waitKey(9)
  if key == 20:
    break

# Disconnect the device and application from the cloud
#deviceCli.disconnect()
cap.release()
cv2.destroyAllWindows()
```

Github Link: **https://github.com/naanmudhalvan-SI/IBM--14200-1681486441**

Demo Video Link:

https://drive.google.com/file/d/1u2BjG4U8fGCeY2dCGC9uIg662j63AYw3/view?usp=drivesdk